



5.6.1 Additional homework ¶

In this chapter, we discussed how the LU factorization (with pivoting) can be used to solve $Ax = y$. Why don't we instead discuss how to compute the inverse of the matrix A and compute $x = A^{-1}y$? Through a sequence of exercises, we illustrate why one should (almost) never compute the inverse of a matrix.

Homework 5.6.1.1. Let $A \in \mathbb{C}^{m \times m}$ be nonsingular and B its inverse. We know that $AB = I$ and hence

$$A \left(\begin{array}{c|ccc} b_0 & \cdots & b_{m-1} \end{array} \right) = \left(\begin{array}{c|ccc} e_0 & \cdots & e_{m-1} \end{array} \right),$$

where e_j can be thought of as the standard basis vector indexed with j or the column of I indexed with j .

1. Justify the following algorithm for computing B :

```

for  $j = 0, \dots, m - 1$ 
    Compute the LU factorization with pivoting :  $P(p)A = LU$ 
    Solve  $Lz = P(p)e_j$ 
    Solve  $Ub_j = z$ 
endfor

2. What is the cost, in flops, of the above algorithm?
3. How can we reduce the cost in the most obvious way and what is the cost of this better algorithm?
4. If we want to solve  $Ax = y$  we can now instead compute  $x = By$ . What is the cost of this multiplication and how does this cost compare with the cost of computing it via the LU factorization, once the LU factorization has already been computed:

```

Solve $Lz = P(p)y$

Solve $Ux = z$

What do we conclude about the wisdom of computing the inverse?

Homework 5.6.1.2. Let L be a unit lower triangular matrix. Partition

$$L = \left(\begin{array}{c|c} 1 & 0 \\ \hline l_{21} & L_{22} \end{array} \right).$$

1. Show that

$$L^{-1} = \left(\begin{array}{c|c} 1 & 0 \\ \hline -L_{22}^{-1}l_{21} & L_{22}^{-1} \end{array} \right).$$

2. Use the insight from the last part to complete the following algorithm for computing the inverse of a unit lower triangular matrix:

```

[L] = inv(L)
L → ( LTL | LTR
      LBL | LBR )
LTL is 0 × 0
while  $n(L_{TL}) < n(L)$ 
    ( LTL | LTR
      LBL | LBR ) → ( L00 | l01 | L02
                      l10T | λ11 | l12T
                      L20 | l21 | L22 )
    l21 :=
    ( LTL | LTR
      LBL | LBR ) ← ( L00 | l01 | L02
                      l10T | λ11 | l12T
                      L20 | l21 | L22 )
endwhile

```

3. The correct algorithm in the last part will avoid inverting matrices and will require, approximately, $\frac{1}{3}m^3$ flops. Analyze the cost of your algorithm.

Homework 5.6.1.3. LINPACK, the first software package for computing various operations related to solving (dense) linear systems, includes routines for inverting a matrix. When a survey was conducted to see what routines were in practice most frequently used, to the dismay of the developers, it was discovered that routine for inverting matrices was among them. To solve $Ax = y$ users were inverting A and then computing $x = A^{-1}y$. For this reason, the successor to LINPACK, LAPACK, does not even include a routine for inverting a matrix. Instead, if a user wants to compute the inverse, the user must go through the steps.

Compute the LU factorization with pivoting : $P(p)A = LU$

Invert L , overwriting L with the result

Solve $UX = L$ for X

Compute $A^{-1} := XP(p)$ (permuting the columns of X)

1. Justify that the described steps compute A^{-1} .
2. Propose an algorithm for computing X that solves $UX = L$. Be sure to take advantage of the triangular structure of U and L .

3. Analyze the cost of the algorithm in the last part of this question. If you did it right, it should require, approximately, m^3 operations.
4. What is the total cost of inverting the matrix?